

Academic Year: 2025-26

LABORATORY MANUAL

Name of the Student:		
Class: TE	Division:	Roll No.:
Subject: Software Laboratory II (2019 Course) [317533]		Exam Seat No.:
Department of Artificial Intelligence and Data Science		

Program Outcomes (PO's):

POs are statements that describe what students are expected to know and be able to do upon graduating from the program. These relate to the skills, knowledge, analytical ability, attitude and behavior that students acquire through the program.

- **PO1: Engineering Knowledge:**

Graduates will be able to apply the Knowledge of the mathematics, science and engineering fundamentals for the solution of engineering problems related to IT.

- **PO2: Problem Analysis:**

Graduates will be able to carry out identification and formulation of the problem statement by requirement engineering and literature survey.

- **PO3: Design/Development of Solutions:**

Graduates will be able to design a system, its components and/or processes to meet the required needs with consideration for public safety and social considerations.

- **PO4: Conduct Investigations of Complex Problems:**

Graduates will be able to investigate the problems, categorize the problem according to their complexity using modern computational concepts and tools.

- **PO5: Modern Tool Usage:**

Graduates will be able to use the techniques, skills, modern IT engineering tools necessary for engineering practice.

- **PO6: The Engineer and Society:**

Graduates will be able to apply reasoning and knowledge to assess global and societal issues

- **PO7: Environment and Sustainability:**

Graduates will be able to recognize the implications of engineering IT solution with respect to society and environment.

- **PO8: Ethics:**

Graduates will be able to understand the professional and ethical responsibility.



- **PO9: Individual and Team Work:**

Graduates will be able to function effectively as an individual member, team member or leader in multi -disciplinary teams.

- **PO10: Communication:**

Graduates will be able to communicate effectively and make effective documentations and presentations.

- **PO11: Project Management and Finance:**

Graduates will be able to apply and demonstrate engineering and management principles in project management as a member or leader.

- **PO12: Life-long Learning:**

Graduates will be able to recognize the need for continuous learning and to engage in life-long learning.



Course Objectives and Course Outcomes (COs)

Course Objectives:

- To understand basic techniques and strategies of learning algorithms.
- To understand various artificial neural network models.
- To make use of tools to solve the practical problems in real field using Pattern Recognition, Classification and Optimization.

Course Outcomes:

On completion of the course, students will be able to–

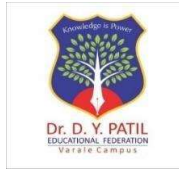
- CO1:** Model Artificial Neural Network, and to analyse ANN learning, and its applications.
- CO2:** Perform Pattern Recognition, Linear classification.
- CO3:** Develop different single layer/multiple layer Perception learning algorithms.
- CO4:** Design and develop applications using neural networks.



Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Department of Artificial Intelligence and Data Science
Academic Year 2025-26



Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Department of Artificial Intelligence and Data Science
Academic Year 2025-26



CERTIFICATE

This is to certify that Mr. /Ms.

of Class TE - AI-DS, Roll No. _____ Examination Seat No. _____

has completed all the practical work in the Software Laboratory II [317533] satisfactorily, as prescribed by Savitribai Phule Pune University, Pune in the academic year 2025-26 (Term-II).

Place:

Date:

Ms. Deepali Narwade
Course In-charge
Department of AI-DS

Dr. Dipannita Mondal
Head
Department of AI-DS

Dr. Suresh Mali
Principal
DYPCOEI, Varale

INDEX

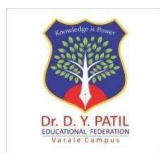
Name of the course: SL-II

Course Code: 317533

Class: TE

Term: II

Sr. No.	Name of the Experiment	Date of Conduction	Date of Checking	Page No.	Sign	Remark
1	Write a Python program to plot a few activation functions that are being used in neural networks.					
2	Generate AND NOT function using McCulloch-Pitts neural net by a python program.					
3	Write a Python Program using Perceptron Neural Network to recognize even and odd numbers. Given numbers are in ASCII form 0 to 9					
4	With a suitable example demonstrate the perceptron learning law with its decision regions using python. Give the output in graphical form.					
5	Write a python Program for Bidirectional Associative Memory with two pairs of vectors.					
6	a. Write a python program to recognize the number 0, 1, 2, 39. A 5 * 3 matrix forms the numbers. For any valid point it is taken as 1 and invalid point it is taken as 0. The net has to be trained to recognize all the numbers and when the test data is given, the network has to recognize the particular numbers					
7	1. Write a python program to show Back Propagation Network for XOR function with Binary Input and Output					
8	Write a python program in python program for creating a Back Propagation Feed-forward neural network					



9	Write a python program to design a Hopfield Network which stores 4 vectors					
10	Write Python program to implement CNN object detection. Discuss numerous performance evaluation metrics for evaluating the object detecting algorithms' performance.					
11	How to Train a Neural Network with TensorFlow/Pytorch and evaluation of logistic regression using tensorflow					
12	TensorFlow/Pytorch implementation of CNN					
13	MNIST Handwritten Character Detection using PyTorch, Keras and Tensorflow					

Name & Signature of Course In-charge

Software Requirements

1. **Operating System**
 - Windows 10 / 11 (64-bit)
 - OR Ubuntu Linux (18.04 or higher)
2. **Programming Language**
 - Python 3.8 or higher
3. **Python IDE / Editor**
 - PyCharm (Community Edition)
 - OR Visual Studio Code
 - OR Jupyter Notebook
 - OR Anaconda Navigator
4. **Required Python Libraries**
 - NumPy – for numerical computation
 - Pandas – for data manipulation
 - Matplotlib – for plotting and visualization
 - Scikit-learn – for machine learning algorithms
 - SciPy – for scientific computing

Hardware Requirements

1. **Processor**
 - Intel Core i3 or higher
 - OR AMD Ryzen 3 or higher
2. **RAM**
 - Minimum: 4 GB
 - Recommended: 8 GB or higher
3. **Hard Disk / Storage**
 - Minimum: 20 GB free disk space
 - Recommended: 50 GB free disk space



Experiment No: 1

1. Write a Python program to plot a few activation functions that are being used in neural networks.

Name of the Student: _____

Class: TE

Batch: T1/T2/T3

Date:

Mark: /10

Signature of the Course In-charge:

Signature of the Head of the department:

GROUP A
EXPERIMENT NO. 1

Practical Title: Plotting Common Activation Functions used in Neural Networks.

Aim: Write a Python program to plot a few activation functions that are being used in neural networks.

Objective:

To learn activation functions that are being used in neural networks and implement in Python.

Theory:

Activation functions are an essential component of artificial neural networks. They introduce non-linearity into the network, enabling it to learn complex patterns and relationships from data. Without activation functions, a neural network would behave like a linear model regardless of the number of layers.

An activation function takes the weighted sum of inputs and bias and transforms it into an output that is passed to the next layer.

Need for Activation Functions

1. Introduce non-linearity
2. Control output range
3. Enable deep networks to learn complex mappings
4. Improve convergence during training

Common Activation Functions

1. Binary Step Function

- Outputs either 0 or 1
- Used in early perceptron models
- Not differentiable, hence not used in deep learning

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

2. Sigmoid Function

- Output range: (0, 1)
- Commonly used in binary classification
- Suffers from vanishing gradient problem

$$f(x) = \frac{1}{1 + e^{-x}}$$

3. tanh (Hyperbolic Tangent) Function

- Output range: (-1, 1)
- Zero-centered, better than sigmoid

$$f(x) = \tanh(x)$$

4. ReLU (Rectified Linear Unit)

- Most widely used activation function
- Efficient and simple
- Can suffer from dying ReLU problem

$$f(x) = \max(0, x)$$

Sample Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the activation functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

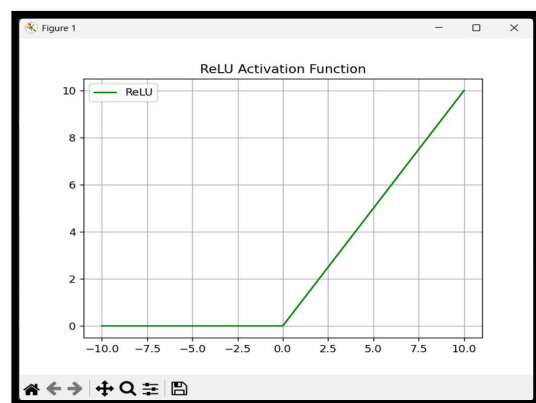
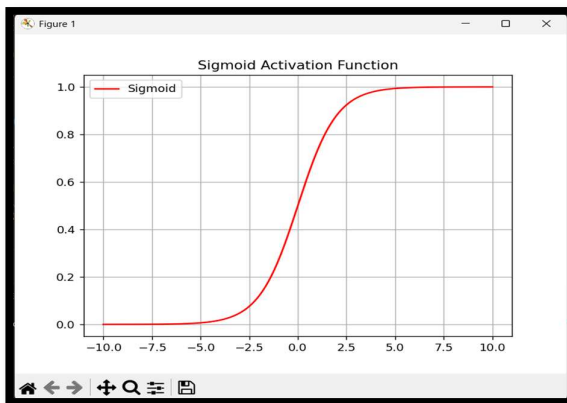
def relu(x):
    return np.maximum(0, x)

# Define the range of values
x = np.linspace(-10, 10, 1000)

# Plot Sigmoid
```

```
plt.plot(x, sigmoid(x), label="Sigmoid", color='r')  
plt.title("Sigmoid Activation Function")  
plt.grid(True)  
plt.legend()  
plt.show()
```

```
# Plot ReLU  
plt.plot(x, relu(x), label="ReLU", color='g')  
plt.title("ReLU Activation Function")  
plt.grid(True)  
plt.legend()  
plt.show()
```



Conclusion:

We have successfully implemented program to plot a few activation functions that are being used in neural networks.

Viva Voce Questions

1. **What is an activation function?**

An activation function is a mathematical function applied to a neuron's input to introduce non-linearity and decide whether the neuron should be activated or not.

2. **Why are activation functions necessary in neural networks?**

They allow neural networks to learn complex, non-linear relationships. Without them, the network would behave like a linear model.

3. **What happens if no activation function is used?**
The neural network becomes equivalent to a single-layer linear model, regardless of the number of hidden layers.
4. **What is the output range of the sigmoid function?**
The output range is between 0 and 1.
5. **Mention one drawback of the sigmoid activation function.**
It suffers from the vanishing gradient problem, making training slow in deep networks.
6. **What is the output range of the tanh function?**
The output range is between -1 and $+1$.
7. **Why is tanh preferred over sigmoid in some cases?**
tanh is zero-centered, which helps in faster convergence during training.
8. **What is ReLU?**
ReLU (Rectified Linear Unit) outputs zero for negative inputs and the input itself for positive values.
9. **What are the advantages of ReLU?**
It is computationally efficient, reduces vanishing gradient problem, and speeds up training.
10. **What is the dying ReLU problem?**
When neurons output zero for all inputs and stop learning due to negative weights.
11. **How does Leaky ReLU solve the dying ReLU problem?**
It allows a small gradient for negative inputs instead of zero.
12. **Which activation function is commonly used in the output layer for binary classification?**
Sigmoid activation function.
13. **Which activation function is preferred for hidden layers in deep networks?**
ReLU or its variants such as Leaky ReLU.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 2

2. Generate AND NOT function using McCulloch-Pitts neural net by a python program.

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the Head: _____

EXPERIMENT NO. 2

Practical Title: Implementation of AND-NOT Function Using McCulloch–Pitts Neural Network.

Aim: Generate AND NOT function using McCulloch-Pitts neural net by a python program.

Theory:

The **McCulloch–Pitts (MP) neuron** is one of the earliest mathematical models of a biological neuron, proposed by Warren McCulloch and Walter Pitts in 1943. It is a **binary threshold neuron** that produces an output based on a weighted sum of inputs and a threshold value.

McCulloch–Pitts Neuron Model

The MP neuron works on the following principles:

- Inputs are binary (0 or 1)
- Weights are fixed
- A threshold decides the output
- Output is binary (0 or 1)

Mathematically,

$$y = \begin{cases} 1, & \sum(w_i x_i) \geq \theta \\ 0, & \text{otherwise} \end{cases}$$

where:

- x_i are inputs
- w_i are weights
- θ is the threshold

AND-NOT Logic Function

The **AND-NOT** function produces output 1 only when:

- First input is 1
- Second input is 0

X1	X2	AND-NOT
0	0	0
0	1	0
1	0	1
1	1	0

MP Neuron Design for AND-NOT

- Input $X_1 \rightarrow$ excitatory input (positive weight)
- Input $X_2 \rightarrow$ inhibitory input (negative weight)
- Weight values:
 - $w_1 = +1$
 - $w_2 = -1$
- Threshold $\theta = 1$

This configuration ensures that output is 1 **only when** $X_1 = 1$ and $X_2 = 0$.

Sample Code

```
import numpy as np

inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
outputs = np.array([0, 1, 0, 0])

W1 = 1
W2 = -1
bias = -0.5

def step_function(x):
    return 1 if x >= 0 else 0

print("Testing McCulloch-Pitts Neural Network for ANDNOT:")
for i in range(inputs.shape[0]):
    input = inputs[i]
    net_input = input[0] * W1 + input[1] * W2 + bias
    output = step_function(net_input)
    print(f"Input: {input[0]} {input[1]} -> Output: {output}")
```

Expected Output:

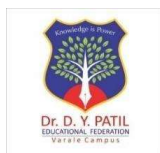
```
PS C:\DeepaliNarwade\Desktop\SL-II Practicals>&
C:/Users/DeepaliNarwade/AppData/Local/Programs/Python/Python313/python.exe
"c:/Users/DeepaliNarwade/Desktop/SL-II Practicals/2p.py"
Testing McCulloch-Pitts Neural Network for ANDNOT:
    Input: 0 0 -> Output: 0
    Input: 0 1 -> Output: 0
    Input: 1 0 -> Output: 1
    Input: 1 1 -> Output: 0
PS C:\Users\DeepaliNarwade\Desktop\SL-II Practicals>
```

Conclusion:

The McCulloch–Pitts neural network successfully implements the **AND-NOT logic function** using appropriate excitatory and inhibitory weights along with a threshold value.

Viva Questions:

1. **What is a McCulloch–Pitts neuron?**
It is a simple artificial neuron model that uses binary inputs, fixed weights, and a threshold to produce binary output.
2. **What type of neuron is the MP neuron?**
A threshold-based binary neuron.
3. **What are excitatory and inhibitory inputs?**
Excitatory inputs increase activation, while inhibitory inputs decrease activation.
4. **Why is a negative weight used for X2 in AND-NOT?**
To inhibit the neuron when X2 is 1.
5. **What is the threshold used in this experiment?**
Threshold value is 1.
6. **Can MP neurons learn weights automatically?**
No, weights are fixed and manually assigned.
7. **What kind of problems can MP neurons solve?**
Linearly separable logic functions.
8. **Why can't MP neurons solve XOR?**
XOR is not linearly separable.
9. **What is the output of AND-NOT when inputs are (1,1)?**
Output is 0.
10. **Are MP neurons used in modern deep learning?**
No, they are mainly of historical and educational importance.



Dr. D. Y. Patil Educational Federation's
Dr. D. Y. PATIL COLLEGE OF ENGINEERING & INNOVATION
Department of Artificial Intelligence and Data Science
Academic Year 2025-26



Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 3

3. Write a Python Program using Perceptron Neural Network to recognize even and odd numbers. Given numbers are in ASCII form 0 to 9.

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 3

Practical Title: Recognition of Even and Odd Numbers Using Perceptron Neural Network (ASCII Digits 0–9).

Aim: Write a Python Program using Perceptron Neural Network to recognize even and odd numbers. Given numbers are in ASCII form 0 to 9.

Theory:

A **Perceptron Neural Network** is the simplest form of an artificial neural network, proposed by **Frank Rosenblatt**. It is a **single-layer supervised learning model** used for binary classification problems.

The perceptron works by computing a weighted sum of inputs and passing it through an activation function (usually a step function) to produce a binary output.

Perceptron Model

$$y = \begin{cases} 1, & \sum(w_i x_i) + b \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Where:

- x_i = input features
- w_i = weights
- b = bias
- y = output

Problem Description

The objective is to **classify digits (0–9) as Even or Odd** using a perceptron neural network.

- Digits are given in **ASCII form**
- ASCII values:
 - '0' → 48
 - '1' → 49
 - ...
 - '9' → 57

Digit	ASCII	Class	Output
0	48	Even	0
1	49	Odd	1
2	50	Even	0
3	51	Odd	1
4	52	Even	0
5	53	Odd	1
6	54	Even	0
7	55	Odd	1
8	56	Even	0
9	57	Odd	1

Learning Rule (Perceptron Learning Rule)

$$w_{new} = w_{old} + \eta(t - y)x$$

Where:

- η = learning rate
- t = target output
- y = predicted output

Sample Code:

```
import numpy as np

class Perceptron:
    def __init__(self, input_size):
        self.weights = np.zeros(input_size)
        self.bias = 0

    def predict(self, x):
        return 1 if np.dot(x, self.weights) + self.bias >= 0 else 0

    def train(self, X, y, epochs=10, lr=0.1):
        for _ in range(epochs):
            for xi, target in zip(X, y):
                prediction = self.predict(xi)
                error = target - prediction
```

```
self.weights += lr * error * xi  
self.bias += lr * error
```

```
X = np.array([[ord(str(i)) % 2] for i in range(10)])  
y = np.array([0 if i % 2 == 0 else 1 for i in range(10)])
```

```
p = Perceptron(1)  
p.train(X, y)
```

```
for i in range(10):  
    result = p.predict([ord(str(i)) % 2])  
    print(f'{i}: {'Even' if result == 0 else 'Odd'}')
```

OUTPUT

```
PS C:\Users\DeepaliNarwade\Desktop\ SL-II Practicals > &  
C:/Users/DeepaliNarwade/AppData/Local/Programs/Python/Python313/python.exe  
"c:/Users/DeepaliNarwade/Desktop/ SL-II Practicals /3p.py"  
0: Even  
1: Odd  
2: Even  
3: Odd  
4: Even  
5: Odd  
6: Even  
7: Odd  
8: Even  
9: Odd  
PS C:\Users\DeepaliNarwade\Desktop\SL-II Practicals>
```

Conclusion:

In this way we have successfully implemented Perceptron Neural Network to recognize even and odd numbers.

Viva Questions:

1. What is a perceptron?

A perceptron is a single-layer neural network used for binary classification.

2. Who proposed the perceptron model?

Frank Rosenblatt.

3. What type of learning is used in a perceptron?

Supervised learning.

4. What is the role of the activation function?

It converts the net input into binary output.

5. What is ASCII?

ASCII is a standard numerical representation for characters.

6. What are ASCII values of digits 0 and 9?

48 and 57 respectively.

7. How are even and odd numbers encoded in this experiment?

Even $\rightarrow$ 0, Odd $\rightarrow$ 1.

8. Can a perceptron solve multi-class problems?

No, it is limited to binary classification.

9. Why can perceptron solve even-odd classification?

Because it is a linearly separable problem.

10. What is the limitation of a single-layer perceptron?

It cannot solve non-linearly separable problems like XOR.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 4

4. With a suitable example demonstrate the perceptron learning law with its decision regions using python. Give the output in graphical form.

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 4

Practical Title: Demonstration of Perceptron Learning Law and Decision Regions Using Python

Aim: With a suitable example demonstrate the perceptron learning law with its decision regions using python. Give the output in graphical form.

Theory:

The Perceptron Learning Law is a supervised learning algorithm used to train a single-layer perceptron for binary classification. It iteratively adjusts weights and bias so that the perceptron correctly classifies training data that is linearly separable.

A perceptron divides the input space into decision regions using a linear decision boundary (a straight line in 2D).

Perceptron Model

For an input vector $\mathbf{x} = [x_1, x_2]$, the output is:

$$y = \begin{cases} 1, & w_1x_1 + w_2x_2 + b \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

Where:

- w_1, w_2 are weights
- b is bias
- y is output class

Perceptron Learning Law

$$w_{new} = w_{old} + \eta(t - y)x$$

$$b_{new} = b_{old} + \eta(t - y)$$

Where:

- η = learning rate
- t = target output
- y = predicted output

The learning process continues until all samples are correctly classified or maximum epochs are reached.

Decision Regions

- A **decision boundary** separates different output classes.
- Points on one side of the boundary belong to class 0.
- Points on the other side belong to class 1.
- In 2D, the boundary is a straight line.

We consider a **binary classification problem** with two input features:

x1	x2	Class
1	1	0
2	1	0
2	3	1
3	2	1

Sample Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Define the input data
X = np.array([[2, 2], [4, 2], [2, 4], [4, 4], [1, 1], [3, 1], [1, 3], [3, 3]])
y = np.array([-1, -1, -1, -1, 1, 1, 1, 1])

# Initialize weights and bias
w = np.zeros(2)
b = 0

# Define the learning rate
learning_rate = 0.1

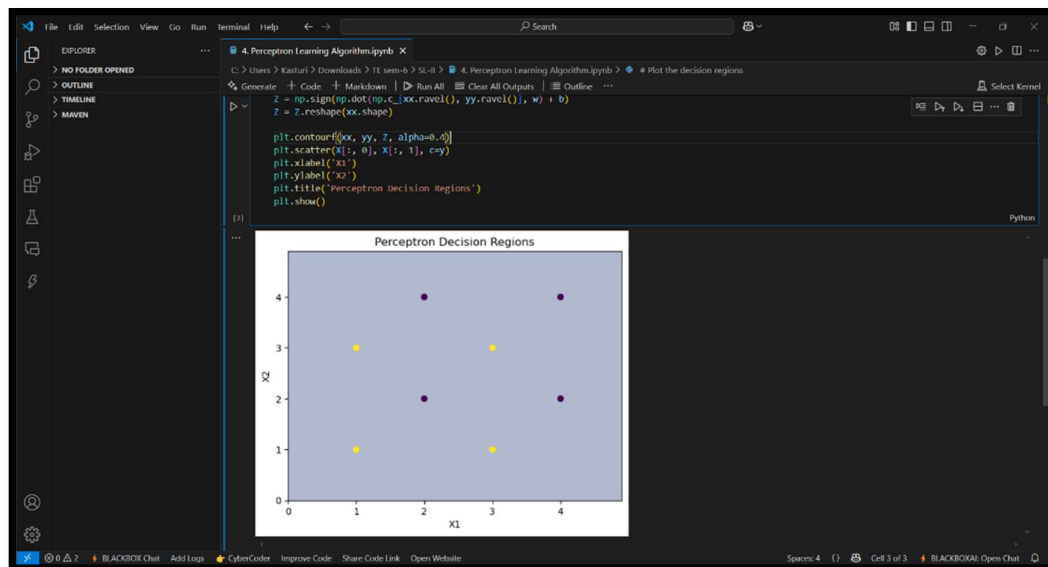
# Define the number of epochs
num_epochs = 10

# Perceptron learning algorithm
for _ in range(num_epochs):
    for i in range(len(X)):
        x = X[i]
        target = y[i]
        activation = np.dot(w, x) + b
        if activation * target <= 0:
            w += learning_rate * target * x
            b += learning_rate * target
```

```
# Plot the decision regions
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1),
                     np.arange(y_min, y_max, 0.1))
Z = np.sign(np.dot(np.c_[xx.ravel(), yy.ravel()], w) + b)
Z = Z.reshape(xx.shape)

plt.contourf(xx, yy, Z, alpha=0.4)
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel('X1')
plt.ylabel('X2')
plt.title('Perceptron Decision Regions')
plt.show()
```

OUTPUT



Conclusion:

Successfully implemented the perceptron learning law with its decision regions using python.

Viva Questions:

1. **What is the perceptron learning law?**
It is a rule to update weights and bias based on classification error.
2. **What type of problems can a perceptron solve?**
Linearly separable problems.
3. **What is a decision boundary?**
A line or surface that separates different classes.
4. **What is the role of learning rate?**
It controls the speed of weight updates.
5. **What happens if the learning rate is too high?**
The model may not converge.
6. **Why is the decision boundary linear?**
Because a single-layer perceptron is a linear classifier.
7. **What activation function is used here?**
Step (threshold) function.
8. **Can perceptron solve XOR problem?**
No, XOR is not linearly separable.
9. **What does bias do in a perceptron?**
It shifts the decision boundary.
10. **Is perceptron still used in deep learning?**
The basic idea is used, but modern networks use multilayer perceptron.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 5

5. Write a python Program for Bidirectional Associative Memory with two pairs of vectors.

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 5

Practical Title: Write a python Program for Bidirectional Associative Memory with two pairs of vectors.

Aim: Write a python Program for Bidirectional Associative Memory with two pairs of vectors.

Theory:

Artificial Neural Networks are computational models inspired by the biological nervous system. They consist of interconnected processing elements (neurons) that work collectively to solve problems such as pattern recognition, classification, and associative memory.

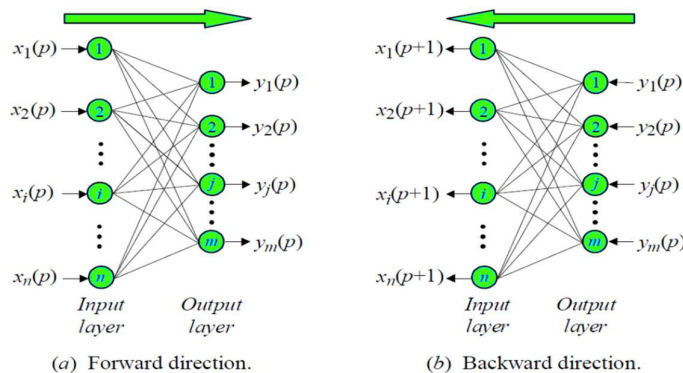
One important class of neural networks is **Associative Memory Networks**, which store patterns and retrieve them when presented with partial or noisy inputs.

Associative Memory

Associative memory is a system that can recall stored information when given a related cue rather than an exact address. There are two types:

- **Auto-associative memory** – input and output patterns are the same.
- **Hetero-associative memory** – input and output patterns are different.

Bidirectional Associative Memory (BAM) is a type of hetero-associative memory.



Bidirectional Associative Memory (BAM)

Bidirectional Associative Memory was introduced by **Bart Kosko**. BAM is a **two-layer recurrent neural network** that stores pairs of patterns and allows recall in **both forward and backward directions**.

Architecture of BAM

BAM consists of:

- **Layer X** with n neurons
- **Layer Y** with m neurons
- Fully connected weights between Layer X and Layer Y
- No intra-layer connections

The network allows:

- Forward mapping: $X \rightarrow Y$
- Backward mapping: $Y \rightarrow X$

Learning in BAM

Learning in BAM is **supervised** and uses the **Hebbian learning rule**.

If:

- X_k = input vector
- Y_k = associated output vector

Then the weight matrix W is computed as:

$$W = \sum_{k=1}^P X_k^T Y_k$$

Where:

- p = number of vector pairs
- X_k^T = transpose of input vector

For backward recall:

$$W^T = \text{Transpose of weight matrix}$$

Recall Process

The recall process is iterative and bidirectional.

Forward Recall ($X \rightarrow Y$):

$$Y = f(XW)$$

Backward Recall ($Y \rightarrow X$):

$$X = f(YW^T)$$

Where:

- $f()$ is the activation function (usually **sign function**)

Activation Function

The commonly used activation function in BAM is the **binary sign function**:

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Advantages of BAM

- Bidirectional recall capability
- Simple learning rule
- Can retrieve stored patterns from partial inputs

Limitations of BAM

- Limited storage capacity
- Sensitive to noise

- Spurious states may occur

Applications of BAM

- Pattern recognition
- Image and signal processing
- Associative data retrieval
- Cognitive modeling

Algorithm (Brief)

1. Define two pairs of bipolar input-output vectors.
2. Initialize the weight matrix to zero.
3. Apply Hebbian learning to calculate the weight matrix.
4. Provide an input vector to Layer X or Layer Y.
5. Perform forward and backward recall.
6. Repeat until stable output is achieved.

Conclusion:

We have successfully Implemented a python Program for Bidirectional Associative Memory with two pairs of vectors.

Viva Questions:

Q1. What is Bidirectional Associative Memory?

BAM is a hetero-associative neural network that stores pattern pairs and allows recall in both forward and backward directions.

Q2. Who proposed BAM?

Bidirectional Associative Memory was proposed by **Bart Kosko**.

Q3. How many layers are present in BAM?

BAM has **two layers**: Layer X and Layer Y.

Q4. Is BAM supervised or unsupervised?

BAM uses **supervised learning**.

Q5. Which learning rule is used in BAM?

BAM uses the **Hebbian learning rule**.

Q6. What type of memory is BAM?

BAM is a **hetero-associative memory**.

Q7. What is the activation function used in BAM?

The **sign function** is commonly used as the activation function.

Q8. What is meant by bidirectional recall?

Bidirectional recall means the network can recall Y from X and X from Y.

Q9. What happens if noisy input is given to BAM?

BAM may still recall the correct associated pattern, but excessive noise may cause incorrect or spurious states.

Q10. What is the weight matrix in BAM?

The weight matrix stores the associations between input and output vectors and is computed using Hebbian learning.

Q11. Is BAM a recurrent neural network?

Yes, BAM is a **recurrent neural network** due to its feedback connections.

Q12. Mention one limitation of BAM.

BAM has limited storage capacity and can produce spurious states.

Q13. Why bipolar vectors are used in BAM?

Bipolar vectors simplify calculations and improve convergence stability.

Q14. What is a spurious state?

A spurious state is an undesired stable state that does not correspond to any stored pattern.

Q15. Give one real-world application of BAM.

Pattern recognition and associative data retrieval.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 6

6. Write a python program to recognize the number 0, 1, 2, 39. A 5 * 3 matrix forms the numbers. For any valid point it is taken as 1 and invalid point it is taken as 0. The net has to be trained to recognize all the numbers and when the test data is given, the network has to recognize the particular numbers.

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 6

Practical Title: Write a Python program to recognize the numbers 0, 1, 2, 3 and 9 using a 5×3 matrix representation.

Aim: Write a python program to recognize the number 0, 1, 2, 3, 9. A 5×3 matrix forms the numbers. For any valid point it is taken as 1 and invalid point it is taken as 0. The net has to be trained to recognize all the numbers and when the test data is given, the network has to recognize the particular numbers.

Theory:

Pattern Recognition

Pattern recognition is a branch of artificial intelligence that focuses on identifying patterns and regularities in data. It involves classification of input data into predefined categories based on extracted features.

Digit recognition is one of the fundamental applications of pattern recognition and neural networks.

Artificial Neural Network for Digit Recognition

Artificial Neural Networks (ANNs) are computational models inspired by biological neurons. They are capable of learning input-output mappings from training examples.

In digit recognition, each digit is represented as a set of pixels, and the neural network learns to associate these pixel patterns with corresponding digit labels.

Pixel-Based Representation of Digits

In this experiment:

- Each digit is represented using a 5×3 matrix
- Total input neurons = 15
- Each element of the matrix is binary:

- **1** → valid (filled) pixel
- **0** → invalid (empty) pixel

This representation converts visual digits into numerical vectors suitable for neural network processing.

Training Data

The digits **0, 1, 2, 3, and 9** are encoded as fixed 5×3 binary matrices.
Each matrix is flattened into a **1×15 input vector**.

These input vectors form the **training dataset**, and their corresponding digit values form the **target outputs**.

Neural Network Model Used

A **single-layer perceptron neural network** is used for classification.

Characteristics of Perceptron:

- Supervised learning algorithm
- Linear classifier
- Consists of:
 - Input layer
 - Weighted sum
 - Activation function

Learning Process

The perceptron learns using a supervised learning approach:

1. Input vector is multiplied by corresponding weights
2. Weighted sum is computed
3. Activation function generates output
4. Error is calculated by comparing output with target

5. Weights are updated iteratively to minimize error

Training continues until:

- All training samples are correctly classified, or
- Maximum iterations are reached

Activation Function

A **step or threshold activation function** is commonly used:

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

This helps in binary classification of digit patterns.

Testing Phase

After training:

- Test input is provided in the same 5×3 format
- The network processes the input
- The trained model predicts the digit based on learned weights

Advantages

- Simple and easy to implement
- Fast training for small datasets
- Suitable for basic digit recognition

Limitations

- Cannot recognize distorted or handwritten digits
- Limited generalization capability
- Works only for predefined patterns

Applications

- Optical Character Recognition (basic)
- Pattern classification
- Educational demonstration of ANN concepts
- Image processing fundamentals

Sample Code

```
import numpy as np
def bipolar(x):
    return np.where(np.array(x) == 0, -1, 1)

def sign(x):
    return np.where(x >= 0, 1, -1)

# Digits 0-3 in 5x3 binary matrix digits = {
"0": [
    [1, 1, 1],
    [1, 0, 1],
    [1, 0, 1],
    [1, 0, 1],
    [1, 1, 1]
], "1": [
    [0, 1, 0],
    [1, 1, 0],
    [0, 1, 0],
    [0, 1, 0],
    [1, 1, 1]
], "2": [
    [1, 1, 1],
    [0, 0, 1],
    [1, 1, 1],
    [1, 0, 0],
    [1, 1, 1]
], "3": [
    [1, 1, 1],
    [0, 0, 1],
    [0, 1, 1],
    [0, 0, 1],
    [1, 1, 1]
]
}
```

```
# Flatten and convert to bipolar X = [ ]
Y = [ ]

for idx, (digit, matrix) in enumerate(digits.items()): flat = [item for row in matrix for item
in row] X.append(bipolar(flat))

# One-hot label in bipolar form label = [-1] * len(digits) label[idx] = 1
Y.append(np.array(label))

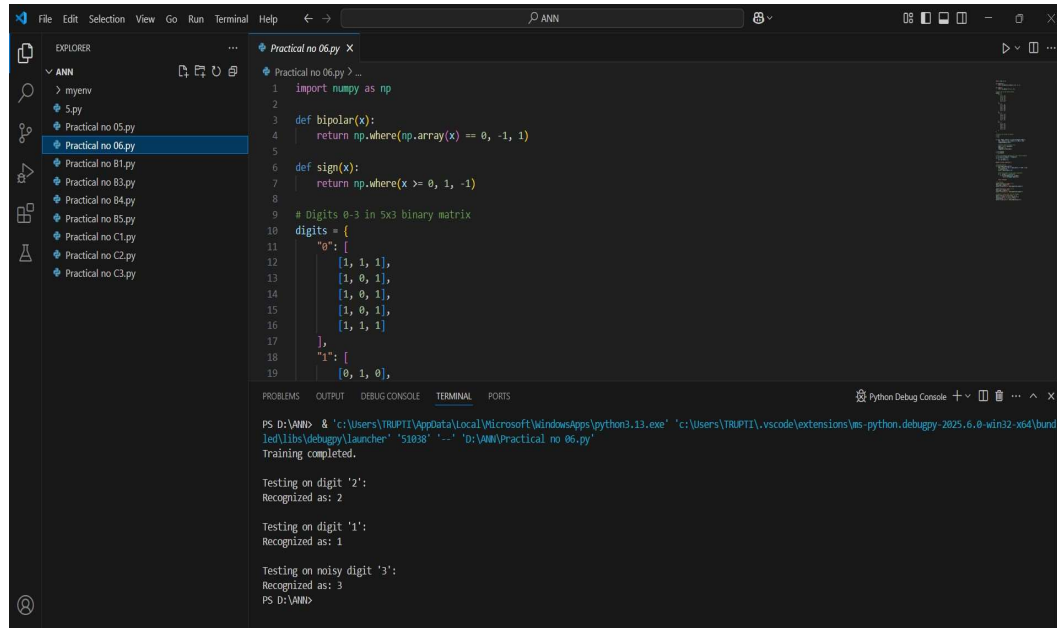
X = np.array(X) Y = np.array(Y)

# Train with Hebbian learning: W = sum of outer products W = np.zeros((X.shape[1],
Y.shape[1]))
for x, y in zip(X, Y): W += np.outer(x, y)
print("Training completed.") # Testing function
def test_digit(input_matrix):
flat = [item for row in input_matrix for item in row] x = bipolar(flat)
y_out = sign(np.dot(x, W))

# Find which digit it most likely represents for i, y_label in enumerate(Y):
if np.array_equal(y_out, y_label): return list(digits.keys())[i]
return "Unknown" # Try testing:
print("\nTesting on digit '2':")
test_input = digits["2"]
print(f'Recognized as: {test_digit(test_input)}')

print("\nTesting on digit '1':") test_input = digits["1"]
print(f'Recognized as: {test_digit(test_input)}')

# Example of noisy input (one bit flipped) print("\nTesting on noisy digit '3':") noisy_3 =
[row[:] for row in digits["3"]]
noisy_3[2][1] = 0 # Introduce noise print(f'Recognized as: {test_digit(noisy_3)}')
```



Conclusion:

Implement Tic-Tac-Toe game where an agent learns to play using reinforcement learning.

Viva Questions:

Q1. What is digit recognition?

Digit recognition is the process of identifying numeric characters from visual or pixel-based input data.

Q2. Why is a 5×3 matrix used in this experiment?

It provides a compact binary representation of digits suitable for neural network input.

Q3. What does a value of 1 represent in the matrix?

It represents a valid or filled pixel.

Q4. What does a value of 0 represent?

It represents an invalid or empty pixel.

Q5. How many input neurons are used?

15 input neurons are used since the input matrix is of size 5×3 .

Q6. Which neural network is used for digit recognition?

A single-layer perceptron neural network is used.

Q7. Is the learning supervised or unsupervised?

The learning is supervised.

Q8. What is the purpose of training the network?

Training adjusts the weights so that the network correctly recognizes the digits.

Q9. What is the role of the activation function?

The activation function decides whether a neuron should be activated based on the weighted sum of inputs.

Q10. Can this network recognize handwritten digits?

No, it can recognize only predefined binary patterns.

Q11. What happens if noisy input is given?

The recognition accuracy may reduce or the output may be incorrect.

Q12. What is meant by flattening the matrix?

Flattening means converting the 5×3 matrix into a single 1×15 input vector.

Q13. What is one limitation of the perceptron model?

It can classify only linearly separable data.

Q14. Is this approach suitable for real-world digit recognition?

No, real-world digit recognition requires advanced models like convolutional neural networks.

Q15. Mention one application of this experiment.

Basic optical character recognition and understanding neural network fundamentals.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 7

7. Write a python program to show Back Propagation Network for XOR function with Binary Input and Output

Name of the Student: _____

Class:

Roll No.:

Batch:

Signature of the Course In-charge: _____

Signature of the HOD: _____

GROUP B

EXPERIMENT NO. 7

Practical Title: Back Propagation Network (BPN) for the XOR function with binary input and output.

Aim: Write a python program to show Back Propagation Network for XOR function with Binary Input and Output.

Theory:

Artificial Neural Networks (ANNs) are computational models inspired by the human brain. They consist of layers of interconnected nodes called neurons. ANNs are capable of learning complex patterns from data using training algorithms.

The XOR (exclusive OR) function is a classic example used to demonstrate the learning capability of multi-layer neural networks because it is not linearly separable. A single-layer perceptron cannot solve the XOR problem, so a **multi-layer network trained using backpropagation** is required.

Back Propagation Network (BPN) is a type of **feedforward neural network** with at least one hidden layer. BPN uses a **supervised learning approach** and adjusts weights by propagating the error backward from the output layer to the hidden and input layers.

The BPN consists of an input layer, one or more hidden layers, and an output layer. Each neuron computes a weighted sum of inputs, applies an **activation function** (usually sigmoid or ReLU), and passes the output to the next layer.

The **backpropagation algorithm** works in the following steps: Forward pass, Error computation, Backward pass, and Weight update.

In the forward pass, inputs are fed to the network, and outputs are computed. The error is calculated by comparing the network output with the desired output. In the backward pass, the error is propagated backward, and the weights are updated using **gradient descent** to minimize the error.

For binary XOR, the input patterns are:

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ 1

1 0 $\rightarrow$ 1

1 1 $\rightarrow$ 0

The network typically has 2 input neurons, 2 neurons in the hidden layer, and 1 output neuron. The **sigmoid function** is used as the activation function to allow differentiation during weight updates.

Advantages of BPN include the ability to learn non-linear functions, generalize from training data, and adapt to complex patterns. Limitations include slow training for large datasets, sensitivity to local minima, and dependence on the choice of learning rate and network architecture.

Applications of BPN include pattern recognition, image classification, function approximation, prediction systems, and solving logical functions like XOR.

Algorithm

1. Initialize network architecture (2 input neurons, 2 hidden neurons, 1 output neuron) and weights randomly.
2. Define XOR input and target output patterns.
3. Set learning rate and number of epochs.
4. For each input pattern, perform forward propagation: compute outputs of hidden and output layers using activation function.
5. Compute output error: difference between network output and target output.
6. Perform backward propagation: calculate error gradients for output and hidden layers.
7. Update weights using gradient descent.
8. Repeat steps 4–7 for all patterns for the given number of epochs until the error converges.
9. Test the trained network with all XOR input patterns to verify outputs.

Conclusion:

Hence we have successfully implemented Back Propagation Network (BPN) for the XOR function with binary input and output.

Viva Questions:

Q1. What is a Back Propagation Network (BPN)?

A BPN is a multi-layer feedforward neural network trained using the backpropagation algorithm to minimize output error.

Q2. Why do we need a hidden layer for XOR function?

Because XOR is not linearly separable, a hidden layer allows the network to learn non-linear decision boundaries.

Q3. What is the role of the activation function?

The activation function introduces non-linearity, enabling the network to learn complex patterns.

Q4. Which activation function is commonly used for XOR problem?

The sigmoid activation function is commonly used.

Q5. What is supervised learning?

Supervised learning is a training method where the network learns from input-output pairs provided as training data.

Q6. How is the error calculated in BPN?

The error is the difference between the target output and the network output.

Q7. What is gradient descent?

Gradient descent is an optimization method used to minimize the error by adjusting network weights in the direction of the negative gradient.

Q8. Can a single-layer perceptron solve XOR?

No, a single-layer perceptron cannot solve XOR because it is not linearly separable.

Q9. What is the typical architecture used for XOR problem?

2 input neurons, 2 neurons in the hidden layer, and 1 output neuron.

Q10. What are the applications of BPN?

Applications include pattern recognition, image classification, function approximation, prediction systems, and solving logical functions like XOR.

Q11. What is overfitting in BPN?

Overfitting occurs when the network memorizes training data but fails to generalize to new inputs.

Q12. How does the learning rate affect training?

A high learning rate can cause instability, while a very low learning rate slows convergence.

Q13. What is the difference between forward and backward propagation?

Forward propagation computes the output from inputs, backward propagation updates weights based on error.

Q14. Can BPN solve only binary functions?

No, BPN can approximate continuous and multi-class functions as well.

Q15. Why is XOR considered a benchmark problem for neural networks?

Because it demonstrates the ability of multi-layer networks to learn non-linear mappings that single-layer perceptrons cannot.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 8

8. Write a python program in python program for creating a Back Propagation Feed-forward neural network

Name of the Student: _____

Class:

Roll No.:

Batch:

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 8

Practical Title:

Implementation of a Back Propagation Feed-forward Neural Network in Python.

Aim:

To implement a feed-forward neural network in Python using backpropagation for learning and to understand the working of multilayer neural networks.

Theory:

Artificial Neural Networks (ANNs) are computational models inspired by the human brain's structure. A feed-forward neural network is the simplest type of ANN where information moves in only one direction—from input to output layer through hidden layers. Each neuron in a layer is connected to every neuron in the next layer with associated weights.

The backpropagation algorithm is a supervised learning method used for training feed-forward neural networks. It minimizes the error between predicted outputs and actual outputs using gradient descent. The algorithm works in two phases: forward propagation and backward propagation. In forward propagation, inputs are passed through the network to compute the output. The error is then calculated, typically using Mean Squared Error (MSE) or Cross-Entropy for classification.

During backward propagation, the network adjusts weights using the error gradient to reduce the loss function. The chain rule of calculus is applied to propagate the error backward from the output layer to the input layer. Learning rate, number of epochs, and the activation function (e.g., sigmoid, ReLU) are crucial parameters for effective training.

Feed-forward networks with backpropagation can solve a variety of problems such as pattern recognition, regression, and classification. However, they require careful tuning to avoid issues like overfitting and vanishing gradients. Python libraries like NumPy or PyTorch simplify implementation by providing matrix operations and automatic differentiation.

Algorithm:

1. Initialize network weights and biases randomly.
2. For each training sample, perform forward propagation to compute output.
3. Calculate the error using the chosen loss function.
4. Perform backward propagation to compute gradients of weights.
5. Update weights using gradient descent.
6. Repeat for all epochs until convergence.

Conclusion:

This lab helps students understand the fundamental concept of supervised learning using neural networks. It demonstrates how a network can learn patterns from data using backpropagation.

Viva Questions:

1. What is the purpose of the activation function in a neural network?
2. Explain the difference between feed-forward and recurrent neural networks.
3. What is gradient descent and why is it used?
4. What are the limitations of backpropagation?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 9

9. Write a python program to design a Hopfield Network which stores 4 vectors

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 9

Practical Title: Design and Implementation of a Hopfield Network in Python

Aim: Write a python program to design a Hopfield Network which stores 4 vectors.

Theory:

A Hopfield network is a type of recurrent neural network (RNN) that serves as an associative memory system. Unlike feed-forward networks, Hopfield networks have connections between all neurons, forming a fully connected graph without self-connections. The network stores patterns in the form of stable states, also called attractors.

The network operates using a symmetric weight matrix derived from Hebbian learning. During recall, the network iteratively updates neuron states to minimize an energy function until it converges to a stored pattern. Hopfield networks are particularly useful for solving optimization problems, pattern recognition, and error correction.

Hopfield networks work well for binary data (0s and 1s or -1s and +1s). The network's dynamics are guided by an energy function, which decreases with each state update until a stable state is reached. This property ensures that the network reliably recalls stored patterns even when presented with noisy or incomplete input.

The limitations include low storage capacity (approximately $0.15 \times$ number of neurons) and susceptibility to spurious states—undesired stable states not corresponding to any stored pattern. Nevertheless, understanding Hopfield networks provides insight into the dynamics of recurrent networks and energy-based neural models.

Algorithm:

Algorithm:

1. Initialize weight matrix using Hebbian learning: $w_{ij} = \sum_p (2x_i^p - 1)(2x_j^p - 1)$ for all stored patterns p .
2. Set initial state of neurons with input pattern.
3. Update neurons asynchronously: $s_i = \text{sign}(\sum_j w_{ij}s_j)$.
4. Repeat until network reaches a stable state (no changes in neuron states).

Conclusion:

This lab illustrates associative memory using Hopfield networks. Students learn how networks can store multiple patterns and recall them despite partial or noisy inputs.

Viva Questions:

1. What is the energy function in a Hopfield network?
2. Explain the concept of attractor states.
3. How is Hebbian learning applied in Hopfield networks?
4. What are spurious states and why do they occur?
- 5.

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	

Experiment No: 10

10. Write Python program to implement CNN object detection. Discuss numerous performance evaluation metrics for evaluating the object detecting algorithms' performance.

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 10

Practical Title: Implementation of CNN for Object Detection in Python

Aim: Write Python program to implement CNN object detection. Discuss numerous performance evaluation metrics for evaluating the object detecting algorithms' performance.

Theory:

Convolutional Neural Networks (CNNs) are specialized deep learning models for processing grid-like data, particularly images. CNNs use convolutional layers to automatically extract spatial features, pooling layers to reduce dimensionality, and fully connected layers for classification. In object detection, CNNs locate and classify multiple objects within an image.

Object detection is more complex than image classification because it requires both localization (bounding box) and classification of objects. Popular architectures include YOLO (You Only Look Once), SSD (Single Shot Detector), and Faster R-CNN. These networks combine CNNs for feature extraction with specialized layers for predicting bounding boxes and class probabilities.

Evaluating object detection performance involves metrics such as **Precision, Recall, F1 Score, Intersection over Union (IoU), Average Precision (AP), and mean Average Precision (mAP)**. Precision measures how many detected objects are relevant, recall measures how many actual objects were detected, and IoU evaluates overlap between predicted and actual bounding boxes. AP summarizes precision-recall trade-offs, while mAP averages AP across all classes.

CNN-based object detection is widely used in autonomous vehicles, surveillance, medical imaging, and robotics. Training involves large labeled datasets like COCO or Pascal VOC, and careful hyperparameter tuning is essential for high accuracy.

Algorithm:

1. Prepare dataset with images and bounding box annotations.
2. Design CNN architecture with convolution, pooling, and fully connected layers.

3. Implement object detection layers (e.g., YOLO head or region proposal network).
4. Train network using labeled dataset and suitable loss function.
5. Evaluate performance using metrics like IoU, AP, and mAP.

Conclusion:

Students gain practical knowledge of CNN-based object detection and learn to measure performance using standard metrics.

Viva Questions:

1. What is Intersection over Union (IoU)?
2. Explain the difference between object detection and classification.
3. Name popular CNN architectures for object detection.
4. What is mean Average Precision (mAP) and why is it important?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 11

11. How to Train a Neural Network with TensorFlow/Pytorch and evaluation of logistic regression using Tensorflow

Name of the Student: _____

Class: **Roll No.:** **Batch:**

Signature of the Course In-charge: _____

Signature of the HOD: _____

GROUP C
EXPERIMENT NO. 11

Practical Title: Training a Neural Network with TensorFlow/PyTorch and Evaluation of Logistic Regression

Aim: How to Train a Neural Network with TensorFlow/Pytorch and evaluation of logistic regression using Tensorflow.

Theory:

Neural networks are powerful computational models that mimic the human brain's functionality for pattern recognition and prediction. Training a neural network involves feeding data into the network, computing predictions, calculating the loss (difference between predicted and actual outputs), and updating weights using optimization algorithms. TensorFlow and PyTorch are two popular frameworks for implementing neural networks, offering automatic differentiation, GPU acceleration, and flexible APIs.

TensorFlow uses a graph-based approach for computation, where operations are defined as nodes in a computational graph. PyTorch, on the other hand, uses dynamic computation graphs, which allows more flexibility for model debugging and experimentation. Both frameworks allow users to define layers, activation functions, loss functions, and optimizers conveniently. A neural network typically consists of an input layer, one or more hidden layers, and an output layer, with nonlinear activation functions such as ReLU or Sigmoid to capture complex patterns.

Logistic regression is one of the simplest neural network models with a single neuron in the output layer using a sigmoid activation function. It is commonly used for binary classification tasks. Training logistic regression involves minimizing the binary cross-entropy loss, which measures the difference between predicted probabilities and actual labels. Evaluation metrics like accuracy, precision, recall, and F1 score are used to assess the model's performance.

When training a neural network, several hyperparameters must be carefully tuned, such as learning rate, number of epochs, batch size, and weight initialization. TensorFlow and PyTorch both support

mini-batch gradient descent, which balances computational efficiency with convergence stability. Monitoring training and validation loss is important to avoid overfitting, and techniques like early stopping and dropout can improve generalization.

In practice, TensorFlow and PyTorch allow seamless integration with datasets and preprocessing pipelines. TensorFlow's `tf.data` API and PyTorch's `DataLoader` make it easy to handle large datasets efficiently. After training, models can be saved and deployed for inference, allowing them to make predictions on new, unseen data.

Algorithm:

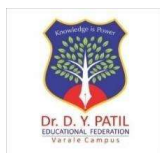
1. Load and preprocess the dataset.
2. Define the neural network architecture (input, hidden, output layers).
3. Select loss function (e.g., binary cross-entropy for logistic regression).
4. Choose optimizer (e.g., Adam, SGD).
5. Train the network over multiple epochs, updating weights using backpropagation.
6. Evaluate performance using metrics such as accuracy, precision, recall, and F1 score.

Conclusion:

This lab introduces students to practical training of neural networks using TensorFlow/PyTorch and reinforces understanding of logistic regression as a foundational model. It demonstrates model evaluation and the importance of tuning hyperparameters for optimal performance.

Viva Questions:

1. What is the difference between TensorFlow and PyTorch?
2. Explain the binary cross-entropy loss function.
3. What is the significance of activation functions in a neural network?
4. How can overfitting be prevented during training?
5. What metrics would you use to evaluate logistic regression performance?



Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 12

12. TensorFlow / Pytorch implementation of CNN

Name of the Student: _____

Class: _____ **Roll No.:** _____ **Batch:** _____

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 12

Practical Title: Convolutional Neural Network Implementation using TensorFlow/PyTorch.

Aim: To implement a CNN using TensorFlow or PyTorch and understand its working for image classification tasks.

Theory:

Convolutional Neural Networks (CNNs) are a type of deep learning model designed specifically for processing data with a grid-like topology, such as images. CNNs automatically extract hierarchical features from input images using layers of convolutions, pooling, and fully connected neurons. Convolutional layers use learnable filters to detect patterns like edges, textures, and shapes, which are essential for recognizing objects within images.

Pooling layers reduce the spatial dimensions of feature maps, decreasing computation and providing spatial invariance. Common pooling techniques include max pooling and average pooling. Fully connected layers at the end of the network perform classification based on the extracted features. Activation functions, typically ReLU, introduce nonlinearity, allowing the network to learn complex relationships in the data.

TensorFlow and PyTorch provide high-level APIs to define CNNs easily. In PyTorch, `nn.Conv2d`, `nn.MaxPool2d`, and `nn.Linear` modules are commonly used, while TensorFlow uses `Conv2D`, `MaxPooling2D`, and `Dense` layers. Training involves computing a loss function (e.g., cross-entropy for classification) and optimizing network weights using backpropagation.

CNNs are widely used in computer vision applications like image classification, facial recognition, and medical image analysis. Performance evaluation involves accuracy, confusion matrices, and sometimes class-wise precision and recall. Hyperparameters like filter size, stride, padding, and number of layers significantly influence model performance and training time.

Implementing CNNs in TensorFlow/PyTorch reinforces understanding of layer architecture, forward propagation, and gradient-based learning. It also familiarizes students with GPU acceleration and batch processing, which are essential for handling large datasets.

Algorithm:

1. Load and preprocess image dataset.
2. Define CNN architecture with convolutional, pooling, and fully connected layers.
3. Choose activation function and loss function.
4. Select optimizer (e.g., Adam, SGD).
5. Train the network over multiple epochs.
6. Evaluate accuracy and other metrics on test data.

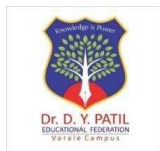
Conclusion:

This lab helps students gain hands-on experience with CNNs, understanding their role in feature extraction and classification. It also demonstrates the differences between TensorFlow and PyTorch implementations.

Viva Questions:

1. Explain the role of convolutional layers in a CNN.
2. What is the purpose of pooling layers?
3. How do ReLU and sigmoid activation functions differ?
4. What are the key hyperparameters in a CNN?
5. How can overfitting be prevented in CNNs?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	



Experiment No: 13

13. MNIST Handwritten Character Detection using PyTorch, Keras and Tensorflow

Name of the Student: _____

Class:

Roll No.:

Batch:

Signature of the Course In-charge: _____

Signature of the HOD: _____

EXPERIMENT NO. 13

Practical Title: Handwritten Digit Recognition on the MNIST Dataset using PyTorch, Keras, and TensorFlow

Aim: To implement handwritten digit recognition using the MNIST dataset across PyTorch, Keras, and TensorFlow, and compare frameworks for performance and ease of use.

Theory:

The MNIST dataset is a widely used benchmark dataset consisting of 70,000 grayscale images of handwritten digits (0–9), with 60,000 images for training and 10,000 for testing. Each image is 28×28 pixels. MNIST is often used for testing and benchmarking machine learning and deep learning models.

Handwritten character detection involves classifying each input image into one of ten categories. Early approaches used feature extraction followed by traditional classifiers, but deep learning, especially CNNs, now achieves near-human accuracy. CNNs automatically learn features directly from pixel values, eliminating manual feature engineering.

PyTorch, Keras, and TensorFlow each provide tools for implementing neural networks. PyTorch offers dynamic computation graphs, Keras provides a high-level API for rapid prototyping, and TensorFlow provides flexibility with both high-level and low-level APIs. These frameworks allow defining layers, activation functions, optimizers, and loss functions, as well as training models with GPU acceleration.

Performance evaluation for MNIST involves measuring accuracy, loss, and confusion matrices. Using different frameworks helps students understand the trade-offs in terms of code simplicity, training speed, and debugging. Hyperparameter tuning (learning rate, batch size, number of epochs, and network depth) is crucial for achieving high accuracy.

MNIST also serves as an introduction to real-world applications such as postal digit recognition, bank check processing, and automated forms digitization. It demonstrates fundamental concepts

like supervised learning, gradient descent, backpropagation, and the importance of normalization in preprocessing.

Algorithm:

1. Load and normalize the MNIST dataset.
2. Define neural network or CNN architecture in PyTorch/Keras/TensorFlow.
3. Select loss function (cross-entropy) and optimizer (e.g., Adam).
4. Train the model using training data over multiple epochs.
5. Evaluate accuracy on test data and analyze misclassified examples.

Conclusion:

This lab provides students with practical experience in implementing handwritten digit recognition across multiple frameworks. It reinforces understanding of CNNs, training pipelines, and evaluation metrics in deep learning.

Viva Questions:

1. Why is MNIST a popular dataset for testing neural networks?
2. Explain the preprocessing steps required before training.
3. What are the differences between PyTorch, Keras, and TensorFlow implementations?
4. What is cross-entropy loss and why is it used for classification?
5. How can misclassifications be analyzed and improved?

Coding Efficiency	Viva	Timely Completion	Total	Dated Sign of Course In-charge
5	3	2	10	